

PRODUCT & ARCHITECTURE PLAN

Asterion

An autonomous engineering organization, not a coding assistant.

Asterion takes a project from a rough idea to deployed, monitored, production software — planning its own work, researching before it acts, building in parallel, and knowing when to hand control back to a human.

A complete, standalone plan: concept • orchestration & coordination • system components • tools & tech stack • operation, market position, and build order.

PART I

Concept

What Asterion is, and the complete lifecycle it runs — from a rough idea to a monitored production system.

[01 · What Asterion Is](#)

[02 · The Full Pipeline, Start to End](#)

01 What Asterion Is

Asterion is a multi-agent system that runs an entire software development lifecycle — not a single task, not a single PR, but the whole loop from a product idea to a monitored production system — with a human holding the controls that actually matter.

In one line

Give Asterion a goal. It scopes it, designs it, plans it, builds it, reviews it, tests it, ships it, watches it in production, and fixes it when something breaks — asking a human only when the decision is genuinely high-stakes.

What it can do today, concretely

- ✓ Turn a one-paragraph feature request into a full architecture, ticket breakdown, and working code
- ✓ Run several independent tickets in parallel instead of one at a time
- ✓ Catch its own mistakes — a failed review or failed test routes straight back to the agent that caused it, automatically
- ✓ Recognize a bug it has already fixed before and reuse the fix instead of re-deriving it
- ✓ Escalate to a human exactly when the risk warrants it, and stay quiet the rest of the time
- ✓ Explain, at any point, why it made a decision — every choice is logged against a reason

02 The Full Pipeline — Start to End

Four phases, thirteen steps. The loop doesn't stop at "code merged" — it closes through production and feeds what it learns back into the next cycle.



The complete lifecycle, grouped by phase — Plan, Build, Ship, Operate — with the feedback loop back into Research and Architecture

WHY THE LOOP MATTERS

Most coding tools stop at "pull request opened." Asterion keeps going through deployment and production monitoring, and what happens in production is fed back into scoping and architecture on the next request — the system gets better at your specific codebase over time, not just better in general.

PART II

Orchestration & Coordination

The layer that actually makes this a system instead of a pile of prompts: who decides what happens next, how work fans out, and how it survives failure at scale.

03 · The Orchestration Layer

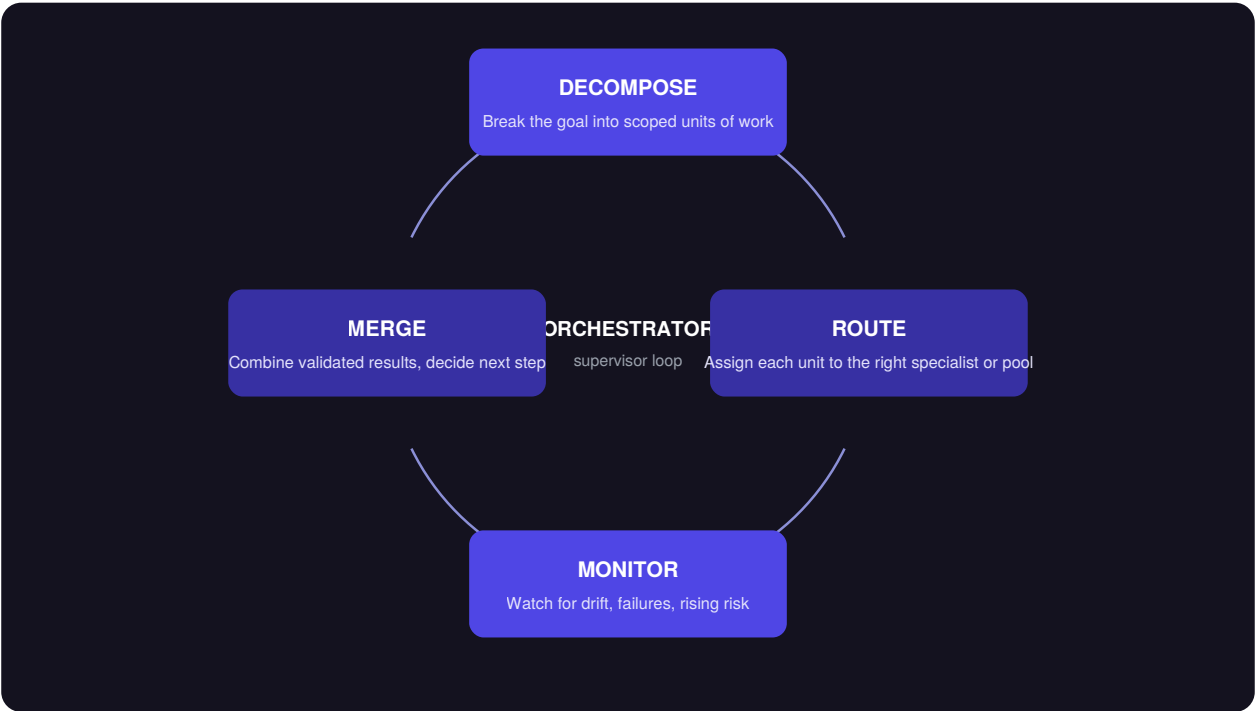
04 · The Core Coordination Pattern

05 · Making It Event-Driven at Scale

06 · Failure Modes Designed Against

03 The Orchestration Layer

Every stage in Section 02 that fans out into specialist work is run by the same orchestrator, doing the same four jobs on a loop. This is the part of Asterion that decides what happens next — nothing else does.



The orchestrator's loop: decompose, route, monitor, merge — repeated at every fan-out point in the pipeline

JOB	WHAT IT ACTUALLY DOES
Decompose	Turns one goal (a ticket, a scope, an incident) into scoped units of work small enough for one specialist to own
Route	Assigns each unit to the right specialist or worker pool — the only place in the system that makes this call
Monitor	Watches for a worker drifting off-task, stalling, or repeating a failure, and intervenes
Merge	Combines validated results and decides the next step — proceed, retry, or escalate to a human

Choosing an orchestration framework

This matters more than it looks — the framework choice determines whether the system has real state checkpointing, resumable human-in-the-loop pauses, and conditional branching, or whether that all has to be hand-built.

FRAMEWORK	ORCHESTRATION MODEL	FIT FOR ASTERION
LangGraph	Directed graph with conditional edges, built-in state checkpointing	Recommended primary. Explicit branching, retries, and resumable human-approval pauses map directly onto the risk-tier gates in Section 08

Temporal

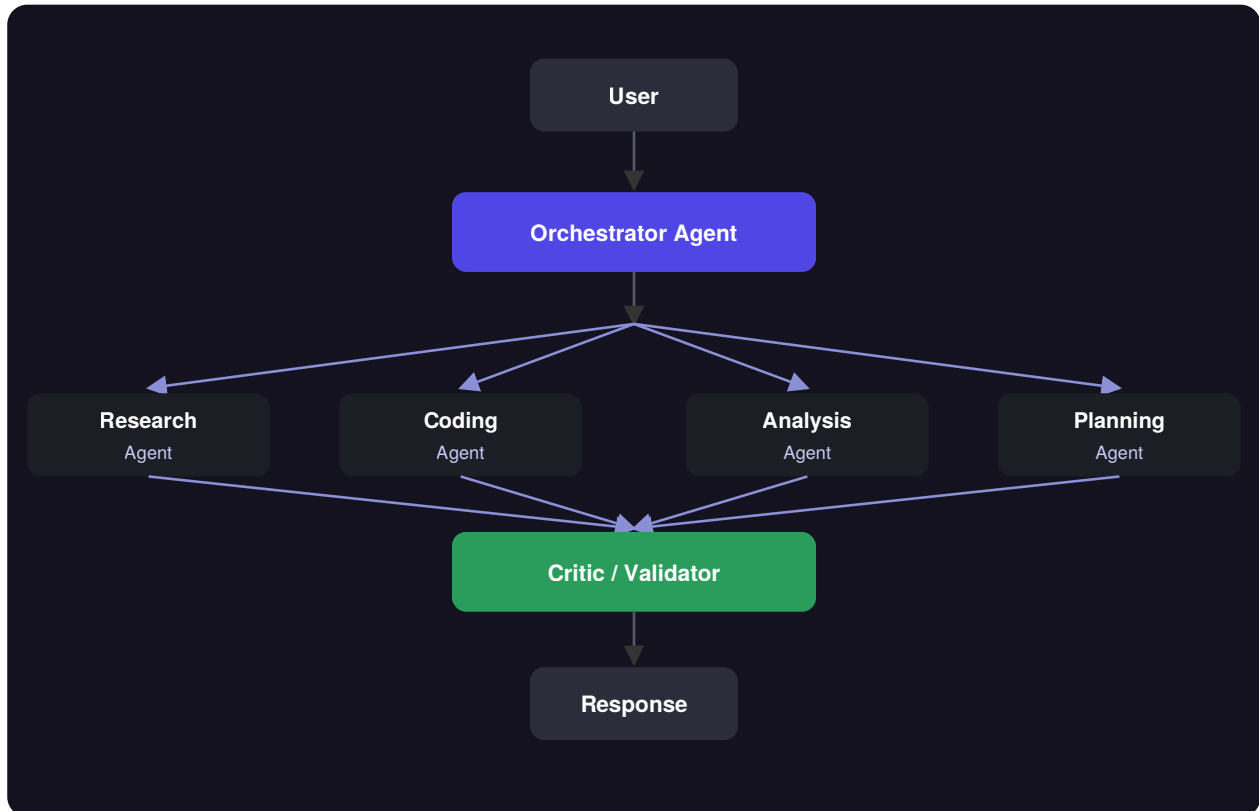
	Durable workflow execution, not agent-specific	Pair underneath LangGraph for crash-safe state and long-running workflows that must survive a restart mid-ticket
CrewAI	Role-based crews, coordination inferred from roles	Useful only for prototyping a new agent's behavior before porting it into the real graph — error handling is too coarse for production
AutoGen / AG2	Conversational GroupChat, agents debate and refine	A good fit specifically inside the Review step — a Coding Agent and a Critic going back and forth is exactly the conversational pattern AutoGen models well

THE HONEST TRADEOFF

Framework debates are a smaller factor than teams expect. The gap between a reliable agent system and a fragile one is almost always the eval pipeline, the observability setup, and the failure-recovery logic — covered in Sections 06 and 09 — not which framework routes the calls.

04 The Core Coordination Pattern

Underneath the pipeline is a single coordination shape, repeated at every stage that fans out into specialist work: one orchestrator, a set of parallel specialists, one validator.



The orchestrator-worker pattern — one accountable coordinator, parallel specialists, one validator before anything ships

This is not a novel shape — it's the same base pattern that orchestration-based coding assistants use internally to decompose a request into subtasks and delegate them to worker agents. What makes it reliable is two design choices baked into it from the start:

- ✓ A single accountable coordinator — no worker decides what happens next, only the orchestrator does
- ✓ A validator between the workers and the response — nothing reaches the user, or the next pipeline stage, unvalidated

WHERE THIS LIVES INSIDE ASTERION

This exact shape repeats at multiple points in the Section 02 pipeline — Scope/Research/Architecture/Planning is one instance of it, and Build/Review/Security/Test is another. The pipeline is really this pattern, applied twice in sequence, with the Orchestration Layer from Section 03 running it both times.

05 Making It Event-Driven at Scale

The diagram in Section 04 is the logical model. Drawn as direct calls, it works for one request at a time. It breaks down once several requests are in flight — the orchestrator ends up managing live connections to every worker, and a worker dying mid-task has nowhere to go. The fix is to keep the exact same shape but route every arrow through an event bus instead of a direct call.



Same coordination shape, implemented as keyed events instead of live connections

DIRECT-CALL VERSION	EVENT-DRIVEN VERSION
Orchestrator holds a connection to every worker	Orchestrator just publishes a task event with a key — no connection to manage
A dead worker stalls its task	The event is replayed from its saved offset by another worker in the group
Adding a worker means the orchestrator's logic changes	Adding a worker means it joins the consumer group — rebalancing is automatic
Every worker gets the full running context	Workers get only the delta since their last task — smaller, cheaper, faster

WHY THIS IS THE RIGHT DEFAULT, NOT JUST A SCALING TRICK

The same key (e.g. the ticket ID) routes every event about that ticket to the same worker instance in sequence, so a worker never has to re-derive context another instance already has — this is what actually eliminates the "reading context again and again" problem, at any scale.

06 Failure Modes Asterion Is Designed Against

Multi-agent systems fail in well-documented, predictable ways. Each one maps to a specific design choice already in Sections 03–05.

FAILURE MODE	WHAT IT LOOKS LIKE	HOW ASTERION AVOIDS IT
Context accumulation	Orchestrator's context grows with every worker's full output until it exceeds the window	Workers return structured results only, not raw reasoning; the Knowledge Store — not the orchestrator's context — holds history
Cost blowup at scale	A workflow that costs cents in testing costs tens of thousands per month at real volume	Delta-only context delivery per event, plus cheaper models on Build/Docs and reasoning-heavy models reserved for Architecture/Debugging
Hot partitions	One ticket type floods a single worker while others sit idle	Keying by ticket ID plus consumer-group rebalancing spreads load automatically as workers are added
Error amplification	A wrong value from one agent is silently trusted and compounds downstream	The Critic/Validator step sits before every merge back into the orchestrator — nothing propagates unchecked
Infinite handoff loops	Agent A hands to B, B to C, C back to A, nobody owns the task	Not applicable to this shape — only the orchestrator decides the next step; workers never hand off to each other directly
Goal drift	A worker wanders from the actual ticket while technically "making progress"	The Supervisor monitors for drift and redirects (Section 03); the Validator catches it before merge regardless

PART III

System Components

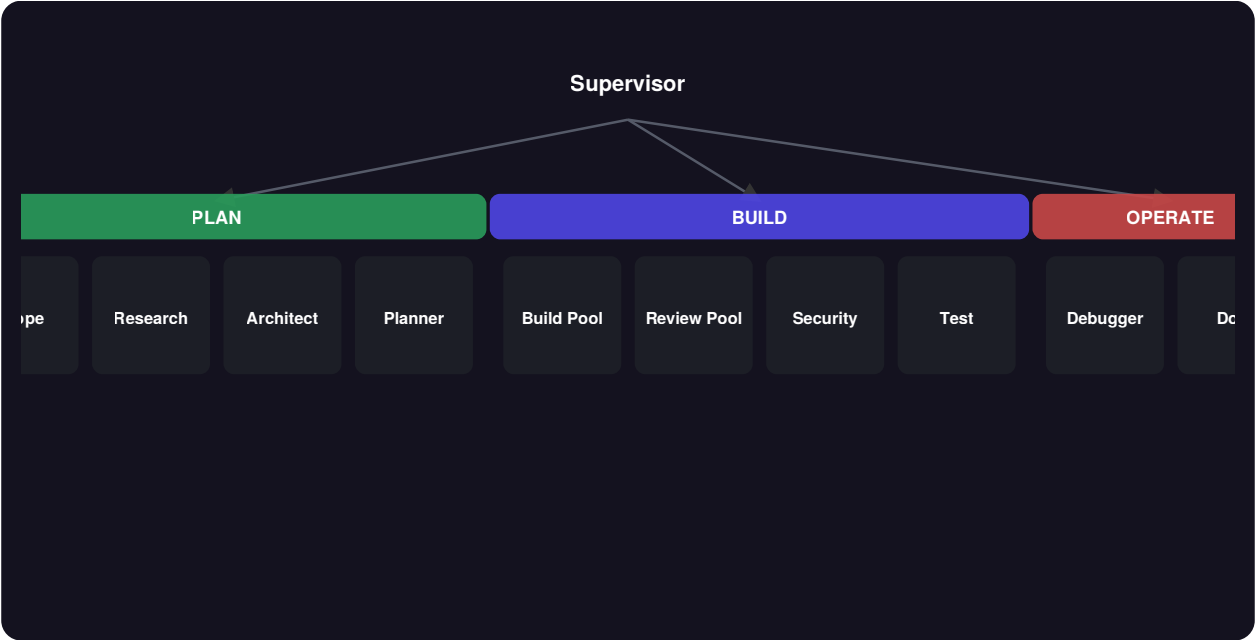
The pieces that give Asterion its roster of specialists, and the memory and autonomy model that governs how much it's trusted to do on its own.

[07 · The Agent Roster](#)

[08 · Memory and Risk-Based Autonomy](#)

07 The Agent Roster

Ten roles, each narrow enough to be reliable, coordinated by one supervisor, grouped by the phase of the pipeline they belong to.



The agent roster, grouped by pipeline phase, all reporting through the Supervisor

AGENT	ROLE
Scope Agent	Reads the request, asks only the questions that actually change the plan, flags risk early
Research Agent	Cross-checks documentation, changelogs, and prior art before any design commitment is made
Architect Agent	Designs the system end-to-end; checks new decisions against past architecture decisions
Planner Agent	Breaks the design into tickets with acceptance criteria, dependencies, and a risk score
Build Pool	Multiple parallel workers, each implementing one ticket at a time
Review Pool	Checks code against architecture and ticket criteria, in parallel with Build
Security Agent	Scans every change for secrets, injection risk, and vulnerable dependencies
Test Agent	Runs automated suites, tracks coverage, flags flaky tests
Debugger Agent	Investigates failures and production incidents using the fix history, proposes or applies a resolution
Docs Agent	Updates documentation the moment a ticket lands, not at the end of a sprint
Supervisor	Routes every event, computes risk scores, watches for worker drift, decides what needs a human

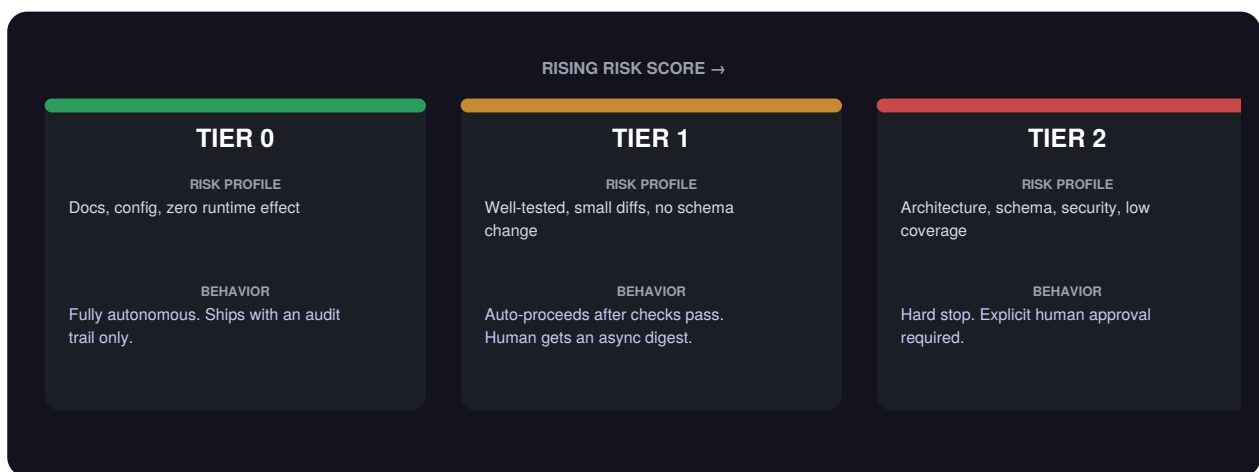
08 Memory and Risk-Based Autonomy

A memory the system actually uses

Asterion keeps a running record of its own codebase structure, its past architecture decisions, every bug it has diagnosed and how it was fixed, and how each agent has performed over time. Every agent consults this record before acting — the Architect checks it isn't contradicting a past decision, the Debugger checks whether this exact failure has happened before.

Risk-scored autonomy instead of blanket approval

Every unit of work carries a computed risk score — blast radius, test coverage of the area touched, and how often that area has broken before. Trivial, well-tested changes proceed on their own with a full audit trail. Anything touching architecture, schemas, security, or poorly-tested code stops for an explicit human decision.



Three autonomy tiers, scored per change and earned per category of work

THE GUARDRAIL

Autonomy is earned per category of work and can only expand with a human's sign-off — it degrades automatically the moment something starts failing more often.

PART IV

Tools & Tech Stack

A researched, current (mid-2026) map of what to actually build this with — category by category, with a specific recommendation and why, not just a list of names.

[09 · Tools We Can Use](#)

[10 · The Recommended Stack](#)

09 Tools We Can Use

Every layer of Asterion maps to an active, production-used category of tooling as of mid-2026. This is the researched option set per layer, with a specific pick and the reason for it.

Orchestration framework

TOOL	BEST FOR	NOTE
LangGraph — recommended	Stateful graphs, checkpointing, human-in-the-loop	Most production-proven of the model-agnostic frameworks; graph model matches the pipeline directly
Temporal	Durable execution, crash-safe retries	Pairs underneath LangGraph rather than replacing it
CrewAI	Fast role-based prototyping	Good for validating a new agent's design before porting to LangGraph; not for production
AutoGen / AG2	Conversational, iterative critique	Fits the Review step's writer/critic dialogue specifically
Claude Agent SDK / OpenAI Agents SDK	Fast path for a single-vendor agent	Consider if committing to one model provider end-to-end

Event bus

TOOL	BEST FOR	NOTE
NATS JetStream — recommended to start	Lightweight, low-ops keyed messaging	Enough for Section 05's task/result topics without running a Kafka cluster
Redis Streams	Simplest possible setup	Fine at small scale; fewer partition/replay guarantees than the alternatives
Kafka	High volume, strict replay and partition guarantees	Move here once ticket volume and audit requirements justify the operational cost

Memory & knowledge store

TOOL	BEST FOR	NOTE
PostgreSQL + pgvector — recommended	ADRs, incident ledger, agent metrics	Zero additional infrastructure if the rest of the stack is already on Postgres
LanceDB	Embedded, serverless vector search	Consider if avoiding a managed database entirely
Code graph index (tree-sitter based)	Structural "what breaks if I change this" queries	Runs alongside pgvector — different query shape (structure, not similarity)

Observability & tracing

TOOL	BEST FOR	NOTE
------	----------	------

LangSmith — recommended given LangGraph	Node-by-node state diffs, full agent graph traces	Deepest integration specifically because Section 03 recommends LangGraph
Langfuse	Self-hosted, open-source, data residency	Best alternative if traces can't leave your infrastructure
Arize Phoenix	OpenTelemetry-native tracing	Best fit if standardizing on OTel across the whole stack, not just agents
OpenTelemetry + Grafana	Whole-stack infra observability	Pairs alongside the LLM-specific tool above — covers hosts, deploys, non-agent services

Security scanning

TOOL	BEST FOR
Semgrep	Static analysis with custom rules — the Security Agent's primary scanner
Snyk	Dependency and CVE scanning, feeds the CVE-response scenario in Section 11
Trivy	Container and image scanning before deploy

Testing & evaluation

TOOL	BEST FOR
Language-native test runners (pytest, etc.)	What the Test Agent actually executes per ticket
Braintrust	Eval-gated CI/CD — blocks a merge if an agent's output regresses against a benchmark set
A held-out ticket benchmark, built in-house	Tracking whether the Build/Debugger agents are actually improving over time, not just passing today's tests

CI/CD & deployment

TOOL	BEST FOR
GitHub Actions (or equivalent)	Triggered directly from <code>code.submitted</code> events
Kubernetes + HPA	Autoscaling the Build/Review worker pools on queue depth
Temporal Workers	Long-running, crash-safe execution of the pipeline itself

Model routing

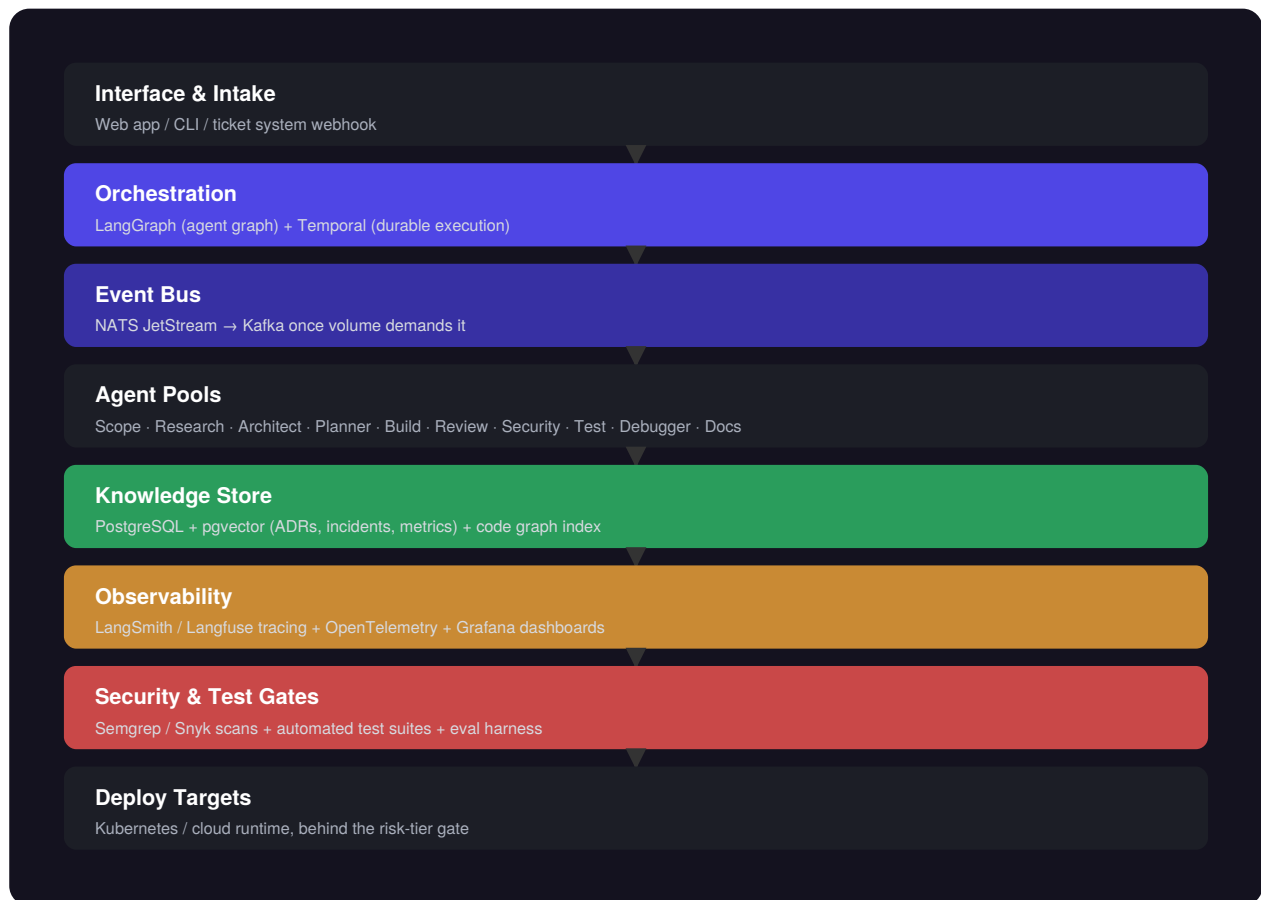
TOOL	BEST FOR
LiteLLM or Portkey	A single gateway across providers — swap models per agent without rewriting call sites

MODEL TIERING, THE ACTUAL PATTERN IN PRODUCTION

Use a fast, cheap model for triage, Build, and Docs agents; reserve the strongest reasoning model for Architecture and Debugger agents, where a wrong call is expensive to unwind. This is a routing decision at the gateway layer, not a framework decision.

10 The Recommended Stack, End to End

Every recommendation from Section 09, laid out as one architecture, top to bottom.



The full recommended stack — from intake to deploy, with observability and security as cross-cutting layers

PART V

Asterion in Operation

What it looks like running, how it compares to what's already on the market, and the order to actually build it in.

[11 · Asterion in Action](#)

[12 · Where the Market Is Today](#)

[13 · What's Actually Different](#)

[14 · Build Order](#)

11 Asterion in Action

Four concrete scenarios, walked through end to end.

FEATURE "Add retry logic to failed payment webhooks"

Scope Agent confirms which failure types should retry and the backoff expectations. Research Agent checks the payment provider's current webhook retry documentation rather than assuming last year's behavior. Architect designs the retry queue. Planner splits it into three tickets. Build Pool implements them in parallel. Security Agent checks the retry path doesn't leak payment payloads into logs. Because the change touches payment code, the risk score routes it to a human gate before deploy.

INCIDENT Production error spike at 2am

Monitoring detects a rising error rate and fires `incident.detected`. Debugger Agent checks the fix history — this exact stack trace was seen four months ago from a dependency upgrade. It proposes the same class of fix, Security and Test agents validate it, and because the fix is narrowly scoped and well-tested, it's within Tier-1 autonomy and ships immediately with a full audit trail. A human sees it in the morning digest, not at 2am.

SECURITY New CVE published for a dependency in use

Security Agent picks this up from a dependency feed, checks the Knowledge Store for every module using the affected package, and opens tickets for each. Because this is security-relevant by definition, it's Tier-2 — the Architect proposes the upgrade path, but a human confirms before anything merges.

LEGACY "Refactor the old billing module, nobody wants to touch it"

Research Agent pulls every past incident and ADR ever logged against that module before the Architect proposes anything — this is exactly the case where assuming instead of checking causes damage. Because historical incident density in that module is high, every ticket in this refactor is automatically routed to Tier-2, regardless of how small each individual diff looks.

12 Where the Market Is Today

As of mid-2026, AI coding tools fall into a few clear categories, and most developers now use more than one.

CATEGORY	EXAMPLES	WHAT THEY'RE BUILT FOR
In-editor agents	Cursor, Windsurf, Zed	Fast, file-aware editing inside an IDE, human driving every session
Terminal-native agents	Claude Code, OpenCode, Aider	Deep, long-running reasoning in a programmable terminal loop
GitHub-native agents	GitHub Copilot's coding agent	Issue-to-PR automation inside existing repos and Actions
Hands-off cloud agents	Devin, Codex Cloud, Cursor Cloud Agents	Fire-and-forget: hand off a task, get back a pull request

Devin is generally regarded as the most autonomous of these — running several agents in parallel cloud sandboxes — but is best suited to well-defined, narrowly-scoped tasks and tends to struggle when requirements are ambiguous or a judgment call is needed mid-task. Claude Code leads on reasoning depth for hard, long-running problems and supports sub-agents and scheduled runs, but operates at the session/repo level rather than governing a full organizational pipeline. Cursor and Copilot optimize for speed and reach rather than end-to-end autonomy. Industry analysis increasingly frames this generation of coding assistants as already using an internal orchestrator-worker decomposition — Asterion's difference is applying that same pattern across the whole SDLC rather than within a single session.

13 What's Actually Different About Asterion

MOST TOOLS TODAY	ASTERION
One agent (or one session) handles a task start to finish	Ten specialized agents, each narrow enough to be reliable, coordinated by a supervisor
Stops at "pull request opened"	Continues through deploy, production monitoring, and incident response
Memory is a static config file per repo	A structured, queryable record of every past decision, bug, and fix — consulted before every action
Autonomy is a single fixed setting (or none)	Autonomy is scored per change and earned per category of work, with automatic downgrade on rising failures
Coordination is direct calls that don't survive a crashed worker	Coordination is keyed events with replay-on-failure, so no single worker dying stalls the system
Human review is uniform — same ceremony for a typo fix and a schema migration	Human attention is reserved for what's actually high-risk; trivial work ships with an audit trail, not a meeting

THE HONEST POSITIONING

Asterion isn't trying to out-code Claude Code or out-autonomous Devin on a single task — those are strong, proven harnesses for exactly that. Asterion is a layer above that question: it's the organization that decides what to build, in what order, how much oversight it needs, and what to do when production breaks at 2am.

14 Suggested Build Order

- 1 Ship the linear pipeline first**
— Scope through Deploy, fully human-gated, direct calls, no event bus yet. This is the trust-building phase.
- 2 Move to the event bus**
once the linear version is stable — same agents, same coordination shape, decoupled per Section 05.
- 3 Turn on the Knowledge Store**
and start logging every decision and incident, even before agents consult it.
- 4 Introduce risk scoring**
once there's enough logged history for it to be meaningful — start Tier-1 autonomy on the single lowest-risk category only.
- 5 Expand autonomy category by category**
, never all at once, always with a human confirming each expansion.

Asterion — plan complete, start to end. Every stage above is buildable independently and ships value on its own before the next stage begins.